

数据结构期末考试编程部分总结

第一题: 大概率为排序

1. 综合问题(包含了可能涉及的算法)

- 用文件输入一些人的信息, 包括姓名, 年龄, 成绩. 文件输入不提供输入元素的个数. 之后完成几项基本的任务.
- Step1: layout all inputs.
- Step2: find the ones who scored 100.
- Step3: count the nums of the ones who are older than 10.
- Step4: reorganize the list according to score, print again.
- 输入样例:

```
ldk 19 90
zh1 18 100
gr1 6 100
lsh 2 95
hsq 20 99
lc 55 60
```

- 输出样例:

```
ldk 19 90
zh1 18 100
gr1 6 100
lsh 2 95
hsq 20 99
lc 55 60
zh1 gr1
zh1
4
zh1 18 100
gr1 6 100
hsq 20 99
lsh 2 95
ldk 19 90
lc 55 60
```

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>
using namespace std;

struct person
{
    string name;
    int age;
    int score;
};

//自定义的一些功能函数，避免重名最好加上my.
void myprint(const person& tmp)
{
    cout << tmp.name << " " << tmp.age << " " << tmp.score << endl;
}
bool myfind(const person& tmp)
{
    return tmp.score == 100;
}
bool mycount(const person& tmp)
{
    return tmp.age > 10;
}
bool mysort(const person& t1, const person& t2)
{
    return t1.score > t2.score;
}

int main() {
    //选用一种比较复杂的输入作为样例
    ifstream ifs;
    ifs.open("in.txt", ios::in);
    vector<person> people;
    string line;
    while (getline(ifs, line)) {
        istringstream iss(line);
        person p;
        iss >> p.name >> p.age >> p.score;
    }
}

```

```

        people.push_back(p);
    }

    //Step1: layout all inputs.
    for_each(people.begin(), people.end(), myprint);
                                                    // 如果调用的是仿函数，需要在

    //Step2: find the ones who scored 100.
    //"for" can be used to find all required elements.
        for (vector<person>::iterator its = people.begin(); its < people.end(); its
        {
            if (its->score == 100) cout << its->name << " ";
        }
        cout << endl;
    // however, "find" as well as "find_if" can only find the first required one.
        vector<person>:: iterator it;
        it = find_if(people.begin(), people.end(), myfind);
        cout << it->name << endl;

    //Step3: count the nums of the ones who are older than 10.
    cout << count_if(people.begin(), people.end(), mycount) << endl;

    //Step4: reorganize the list according to score, print again.
    sort(people.begin(), people.end(), mysort);
    for_each(people.begin(), people.end(), myprint);
        //如果不是自定义的数据类型，直接用greater<int>(),less<int>()就行.

    ifs.close();
    return 0;
}

```

2. 简单运算可能涉及的重载运算符

```

#include<iostream>
#include<algorithm>
#include<string>
using namespace std;
class person
{
public:
    string name;
    int age;

    /*构造函数的重载*/
    person(){}
    person(string n, int a): name(n), age(a) {}

    /*+运算符重载*/
    person operator+(const person& p)
    {
        person together;
        together.age = p.age + this->age;
        together.name = p.name + " " + this->name;
        return together;
    }

    /*前置++运算符重载*/
    person& operator++()
    {
        this->age ++;
        return *this;
    }

    /*后置++运算符重载*/
    person operator++(int)
    {
        person tmp = *this;
        this->age ++;
        return tmp;
    }

    /*函数引用运算符重载，也称为"仿函数"*/
    string operator()(string tmp)
    {
        string t = this->name + tmp;
        return t;
    }

```

```

    }
};

/*左移运算符只能在类之外进行重载，ostream必须是p*/
ostream& operator<<(ostream& cout, const person& p)
{
    cout << "name:" << p.name << "; age:" << p.age << ";" << endl;
}

int main()
{
    person a("ldk", 19), b("zh1", 18);
    person c;
    c = a + b;
    cout << c++ << endl;
    cout << ++c << endl;
    cout << c(" go go go!") << endl;
    return 0;
}

```

第二题: 大概率为树

1. 链式树的构建和基本要素:

- 利用括号法进行树的构建.
- 利用括号法进行树的显示 - 先根遍历.
- 查询某一给定节点(如输出孩子的值)
- 查询全部叶子结点(或者分支节点).
- 查询树的高度.
- 查询祖先节点.....
- 层次遍历(包括输出每层的节点个数).
引入每层的last变量.
- 满二叉树, 完全二叉树的判定....

```
struct node
{
    int data;
    node* lc;
    node* rc;
    node(): lc(NULL), rc(NULL){}
    node(int num): data(num), lc(NULL), rc(NULL){}
};
```

```

class tree
{
    public:
        node* root;
        tree(): root(NULL){}

        // 利用括号法进行树的构建.
        void create(string str)
        {
            stack<node*> t;
            bool flag;
            node* q;
            int i = 0, count = 0;
            while(i < str.length())
            {
                switch(str[i])
                {
                    case '(':
                        flag = true;
                        t.push(q);      break;
                    case ',':
                        flag = false;  break;
                    case ')':
                        t.pop();        break;
                    default:
                        int count = 0;
                        while (str[i] >= '0' && str[i] <= '9')
                        {
                            count = count*10 + str[i] - '0';
                            i++;
                        }
                        i--;
                        q = new node(count);
                        if(root == NULL)
                            root = q;
                        else{
                            if(flag){
                                if(!t.empty()) t.top()->lc = q;
                            }else{
                                if(!t.empty()) t.top()->rc = q;
                            }
                        }
                        break;
                }
                i++;
            }
        }
    };
}

```

```

    }
}

//先序和中序确定树的结构.
void create2(vector<int>& first, vector<int>& middle)
{
    root = create22(first, 0, middle, 0, first.size());
}
node* create22(vector<int>& first, int i, vector<int>& middle, int j, int n)
{
    if(n <= 0) return NULL;
    int tmp = j;
    while(middle[tmp] != first[i]) tmp++;
    node* p = new node(first[i]);
    int k = tmp - j;
    p->lc = create22(first, i+1, middle, j, k);
    p->rc = create22(first, i+1+k, middle, tmp+1, n-k-1);
    return p;
}

//利用括号法进行树的显示 - 先根遍历.
void display()
{
    display2(root);
}
void display2(node *b)
{
    if (b!=NULL)
    {
        cout << b->data;
        if (b->lc!=NULL || b->rc!=NULL)
        {
            cout << "(";
            display2(b->lc);
            if (b->rc!=NULL)
                cout << ",";
            display2(b->rc);
            cout << ")";
        }
    }
}

//后序遍历进行输出.

```



```

void ass()
{
    ass2(root);          cout << endl;
}
void ass2(node* root)
{
    if(root ==NULL) return;
    else{
        if(root->lc != NULL) ass2(root->lc);
        if(root->rc != NULL) ass2(root->rc);
        cout << root->data << " ";
    }
}

//查询树的高度.
int height()
{
    return height2(root);
}
int height2(node* r)
{
    if (r == NULL) return 0;
    else return 1 + max(height2(r->lc), height2(r->rc));
}

//查询全部叶子结点(或者分支节点).
void leafprint()
{
    leafprint2(root);
}
void leafprint2(node* root)
{
    if(root == NULL) return;
    else if(root->lc == NULL && root->rc == NULL)
    {
        cout << root->data << " " ; return;
    }
    leafprint2(root->lc);
    leafprint2(root->rc);
}

//层次遍历.
void level()

```

```

{
    queue<node*> st;
    st.push(root);
    while(!st.empty())
    {
        cout << st.front()->data << " ";
        if (st.front()->rc != NULL) st.push(st.front()->rc);
        if (st.front()->lc != NULL) st.push(st.front()->lc);
        st.pop();
    }
    cout << endl;
}

//查询某一给定节点(如输出孩子的值)
void findkids(int a)
{
    node *getone;
    bool decide = findkids2(root, getone, a);
    if(decide){
        if(getone->lc != NULL) cout << "左孩子:" << getone->lc->data << endl;
        else cout << "左孩子不存在!" << endl;
        if(getone->rc != NULL) cout << "右孩子:" << getone->rc->data << endl;
        else cout << "右孩子不存在!" << endl;
    }else cout << "失败!" << endl;
}

bool findkids2(node* root, node*& getone, int a)
{
    if(root == NULL) return false;
    else{
        if (root->data == a){
            getone = root;
            return true;
        }else{
            if(findkids2(root->lc, getone, a)) return true;
            if(findkids2(root->rc, getone, a)) return true;
            return false;
        }
    }
}

};

```

2. 需要掌握的其他常用树结构:

i. 双亲存储.

ii. 顺序存储二叉树.

3. 树最多能考到的难点:

i. 先序和中序确定树的结构.

代码见上.

ii. 哈夫曼树的构建(求节点的哈夫曼的带权路径长度).

作业题.

4. 其他难度合适的考点:

i. 约瑟夫问题. 链表结构需要手敲一遍, 包括构造, 输出, 插入, 删除, 查找等. 直接用循环双链表进行练习.

ii. 表达式求解问题.

iii. 迷宫问题, 用一种方法实现.

iv. 字符串查找问题, BF和KMP.

v. 图: 邻接矩阵与拓扑排序, 深度和广度优先遍历.

vi. 排序树.

vii. 平衡二叉树. (难度较大)